

REPORT

On

Smart Minesweeper

*Submitted in Partial Fulfillment of the
Requirements for the Degree*

of

Bachelor of Technology

in

Computer Science Engineering

As part of “**Artificial Intelligence (CSE3705)**” course

By

GROUP-1

Harsh Singh

Anjali

Sunny

Nishant Kasana

Under The Supervision Of

Dr. Anantha Rao

SCHOOL OF ENGINEERING AND TECHNOLOGY



**BML MUNJAL
UNIVERSITY™**

FROM HERE TO THE WORLD

BML Munjal University, Gurugram- 122413

NOVEMBER 2024

Candidate's Declaration

We hereby certify that the work on the project entitled, "Smart Minesweeper ", in partial fulfilment of requirements for the award of Degree of Bachelor of Technology in School of Engineering and Technology at BML Munjal University, having University Roll No.1232434, is an authentic record of my own work carried out during a period from July 2024 to December 2024 under the supervision of Dr. Anantha Rao

Harsh Singh(220482)

Anjali(220659)

Sunny(220717)

Nishant Kasana(220713)

Supervisor's Declaration

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Faculty Supervisor Name: Dr. Anantha Rao Signature:

Acknowledgement

We are highly grateful to Dr. Anantha Rao, BML Munjal University, Gurugram, for providing supervision to carry out the seminar/case study from July-December 2024. Dr. Anantha Rao has provided great help in carrying out our work and is acknowledged with reverential thanks. Without wise counsel and able guidance, it would have been impossible to complete the training in this manner.

We would like to express thanks profusely to thank Dr. Anantha Rao, for stimulating us from time to time. We would also like to thank the entire team at BML Munjal University. We would also thank our friends who devoted their valuable time and helped our in all possible ways toward successful completion.

Harsh Singh(220482)

Anjali(220659)

Sunny(220717)

Nishant Kasana(220713)

Contents

Candidate's Declaration.....	2
Supervisor's Declaration.....	2
Acknowledgement.....	3
Abstract.....	Error! Bookmark not defined.
1. Introduction.....	Error! Bookmark not defined.
2. Literature Review.....	Error! Bookmark not defined.
3. Methodology.....	Error! Bookmark not defined.
4. Analysis and Discussion of Results.....	Error! Bookmark not defined.
5. Conclusion.....	Error! Bookmark not defined.
6. References/Bibliography.....	Error! Bookmark not defined.
7. Plagiarism Check Report.....	Error! Bookmark not defined.

Abstract

This project, titled "Smart Minesweeper," aims to develop an artificial intelligence agent capable of efficiently solving the popular Minesweeper game. The Minesweeper problem is modeled as a constraint satisfaction problem (CSP), where decisions are based on partial knowledge of the environment. The agent employs a combination of techniques, including a basic solver for immediate constraints, a subset solver for inferring relationships between equations, and a heuristic-driven random solver for uncertain scenarios. Performance evaluations across different board sizes and mine densities demonstrate the agent's ability to flag mines and open cells with high accuracy, particularly in low to moderate mine densities. The project highlights the importance of optimized knowledge base handling and demonstrates how incorporating advanced techniques like backtracking could further improve decision-making. This work not only contributes to AI research in CSPs but also provides insights into decision-making strategies for systems operating under uncertainty.

Introduction

This project is intended to explore how data collection and inference can inform future action and future data collection. This is a situation frequently confronted by artificial intelligence agents operating in the world - based on current information, they must decide how to act, balancing both achieving a goal and collecting new information. Additionally, this project stresses the importance of formulation and representation. There are a number of roughly equivalent ways to express and solve this problem, it is left to you to decide which is best for your purposes.

Literature Review

Minesweeper, as a constraint satisfaction problem, has been studied extensively within the domain of artificial intelligence. Existing approaches often rely on logical inference, heuristic evaluation, and probabilistic reasoning to navigate the uncertainties of the game. Research on SAT solvers and probabilistic graphical models, such as Bayesian networks, has been pivotal in addressing similar problems. For example, heuristic-based algorithms have been used to prioritize cell openings based on calculated risks, while subset solving techniques have been employed to derive logical conclusions from overlapping constraints. The use of CSPs in broader AI contexts, such as planning and optimization, provides a foundational framework for this project. Studies in reinforcement learning and decision theory also offer insights into improving agent performance in high-density mine scenarios. This project builds upon these methods by integrating multiple solving strategies into a unified framework, enhancing the agent's ability to make informed decisions and adapt to dynamic conditions.

Methodology

The Smart Minesweeper agent is designed with a modular approach, incorporating three primary solving phases. The **basic solver** identifies and acts on immediate constraints, such as cells that are guaranteed to be safe or contain mines, based on straightforward logical deductions. The **subset solver** extends this capability by

analyzing relationships between equations, deducing new information through subset evaluation. For instance, it resolves equations like $A+B+C=2$ + $B + C = 2$ + $A+B+C=2$ and $B+C=1$ + $C = 1$ + $B+C=1$ to infer AAA as a mine. When neither approach yields actionable insights, the agent employs a **heuristic-driven random solver**, which calculates probabilities for each unopened cell and prioritizes those with the lowest risk. The agent's knowledge base, implemented as a dynamically updated list of constraints, enables efficient information management and decision-making. Performance metrics, such as accuracy, time efficiency, and success rates across varying mine densities, are used to evaluate the agent's effectiveness.

1.1 Representation

- **Code:** While writing the code, the minesweeper game is represented as a numpy matrix. We have terminal based input variables through which we can change the size of the matrix (default size = 8x8) and the mine density. Being a constraint-satisfaction problem, we model the constraint equations as tuples - (*list of variables, equation value*).
- **Visualisation:** Figure 1 displays our visualisation of the minesweeper environment. The left matrix shows the actual map of minesweeper generated from our code. The right side is a copy of the left matrix, the only difference is that the agent of our model is run on the right matrix. So as the agent makes decisions, the right plot changes - safe cells are opened, the number displayed and mines are flagged. The left actual plot helps us test the efficiency and accuracy of our agent. The information is represented just like a normal minesweeper game, through numbers, dots(where mines are placed) and flags(where our agent thinks a mine is).

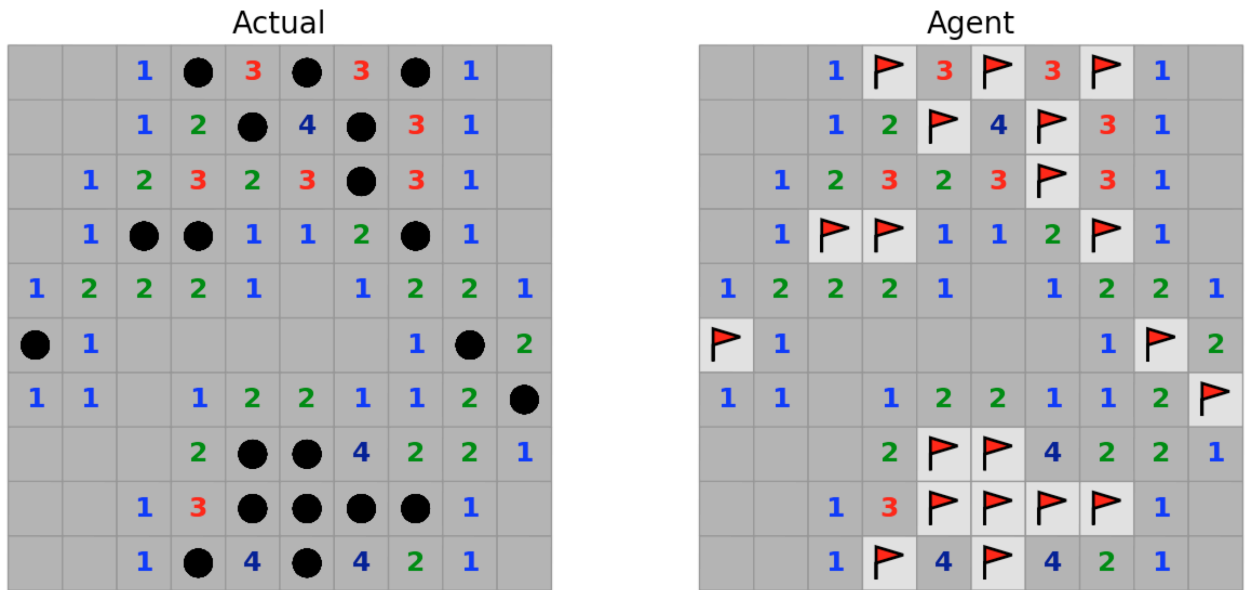


Figure 1: Minesweeper Game Visualisation

1.2 Inference

Our agent keeps 2 lists:

- mine-variables - a list of all cells/blocks which it thinks as having mines.
- non-mine-variables - a list of all safe

cells/blocks The agent plays the game in 3 phases recursively.

- **Basic solver:** The task of a basic-solver is obvious from its name - to solve the board just by looking at the cells without any equations solving or subset evaluation.
 - **Click all non-mine-cells:** By default, we have coded the environment in such a way that the first cell which the agent opens/clicks is not a mine. So the (0, 0) cell is always a non-mine variable. The agent iterates through all the variables in the non-mine-variables list and opens them. When a cell is opened by the agent, the following actions are taken by it
 - * **Form the constraint-equation for the variable:** When a non-mine-variable is opened, a constraint-equation is formed which consists of addition of its neighbours and an equation value.
 - * **Remove the variable from other equations:** Since a cell is opened so it becomes harmless. Hence it has to be removed from all the constraint- equations of other variables in which it occurs
 - **Flag all mine cells:** If the number of variable in equation is equal to value then any variable in the list mine-variables is a potential mine and has to be

flagged.

The agent flags all the cells in this list one by one. After flagging the cell it removes the cell from all the constraint-equations it is present in. However, there is a major difference here as compared to a non-mine cell. We do not only remove a mine-variable from the equation but also subtract the value of the equation, since now it contains one less mine-variable.

- **Check the list of equations for more mine-variables and non-mine variables:** This is the step which yields more mine and non-mine variables for the agent to flag and open respectively. After removing non-mine variables and min variables from other equations, the agent checks its knowledge base for the following two types of equations
 - * $A + B + C = 0$: Any equation with a value of 0 gives us non-mine-variables. Here A, B and C are all non-mine variables.
 - * $A + B + C = 3$: Any equation where the number of variables in the equation is equal to the equation value gives us mine variables. Here A, B and C are all mine variables.
- **Subset solver:** There will be a time when the above basic solver can no longer find individual equations that yield more non-mine and mine variables. This is where we try breaking down subsets of equations. The agent creates subsets of constraint equations present in the knowledge base and starts solving them. For example: if there are two equations $A + B + C = 2$ and $B + C = 1$, then the agent deduces that A definitely has a bomb by substituting the second equation into the first one. This procedure is run iteratively and it solves the knowledge base quite efficiently.
- **Random opening solver:** Finally there will be a time when no further subsets can be deduced and all the equations are exhaustive of each other. For example: if there are two equations $A + B + C + D = 3$ and $B + C = 1$, solving the subset does not lead to any new inferences. In such a scenario, the agent relies upon random (with a heuristic) opening of cells to make and deduce new equations. If the agent comes to this stage, it means that the current knowledge base is not sufficient enough to make any further inferences. The next cell selection is not totally random. The agent uses a heuristic based on the probability of any cell having a bomb before making a random click. The lesser the value of this heuristic, the more likely it is to open that cell.

Based on the three phases the agent goes through, the agent deduces almost everything possible before continuing further. But, the agent can be improved by incorporating a backtracking algorithm. With this incorporation, we can ensure that the agent has been thorough by inferring all possible inferences from the knowledge base. This would ensure that the agent is extensive in solving the minesweeper. Our agent is able to deduce everything from a clue before proceeding because from the way we have coded our agent, it will try to break down equations to either of the two forms we described previously. If there are no such equations it will store them and proceed to getting more clues. We are sure about this because we also have a way of visualising the knowledge base of our minesweeper agent

```
#####
[(2, 0), (2, 1)] = 1
[(0, 2), (1, 2), (2, 0), (2, 1), (2, 2)] = 2
#####
[(2, 0), (2, 1)] = 1
[(2, 0), (2, 1), (2, 2)] = 2
[(0, 3), (1, 3), (2, 1), (2, 2), (2, 3)] = 2
#####
[(2, 0), (2, 1)] = 1
[(2, 0), (2, 1), (2, 2)] = 2
[(2, 1), (2, 2), (2, 3)] = 2
[(0, 4), (1, 4), (2, 2), (2, 3), (2, 4)] = 1
#####
[(2, 0), (2, 1)] = 1
[(2, 0), (2, 1), (2, 2)] = 2
[(2, 1), (2, 2), (2, 3)] = 2
[(2, 2), (2, 3), (2, 4)] = 1
#####
[(2, 0), (2, 1)] = 1
[(2, 0), (2, 1), (2, 2)] = 2
[(0, 6), (1, 6)] = 1
[(0, 6), (1, 6), (2, 6)] = 1
[(2, 2), (3, 2), (3, 3), (3, 4)] = 1
#####
[(4, 3), (4, 4), (4, 5)] = 1
[(4, 4), (4, 5), (4, 6)] = 1
[(0, 6), (0, 7), (1, 7), (2, 7)] = 1
[(2, 7), (3, 7), (4, 5), (4, 6), (4, 7)] = 1
#####
[(4, 5), (4, 6)] = 1
[(4, 5), (4, 6), (4, 7)] = 1
[(3, 0), (3, 1)] = 1
[(3, 1), (4, 1), (5, 1), (5, 2), (5, 3)] = 2
[(4, 5), (5, 3), (5, 4), (5, 5)] = 1
#####
[(5, 5), (5, 6), (5, 7)] = 1
[(5, 6), (5, 7), (5, 8)] = 1
[(3, 9), (4, 9), (5, 7), (5, 8), (5, 9)] = 1
#####
[(5, 6), (5, 7)] = 1
[(5, 6), (5, 7), (5, 8)] = 1
[(5, 7), (5, 8), (5, 9)] = 1
[(5, 0), (6, 0), (7, 0), (7, 1), (7, 2)] = 2
[(7, 1), (7, 2), (7, 3)] = 1
[(7, 2), (7, 3), (7, 4)] = 1
#####
```

Figure 2: Printing all the equations in the knowledge base

1.3 Decision

Given the current state of the board, and a state of knowledge about the board - the knowledge base - the agent uses the following 3 steps to move ahead

- Check the list of mine and non-mine variables using basic solver. If any mines and non-mines found, then flag and open them respectively. If nothing can be deduced, proceed to evaluating subsets.
- Solve subsets and find non-mines and mines by breaking down the equations.
- Click randomly if no subsets can be resolved.

There is one potential risk that is bound to occur in the third phase since the agent opens cells in a quasi-random manner. Although the heuristic reduces the risk, it does not guarantee the opening of a safe cell. As mentioned earlier, this can be tackled using a backtracking method. Another risk/disadvantage is that in environments with high-mine density the heuristic-method of randomly opening cells would fail.

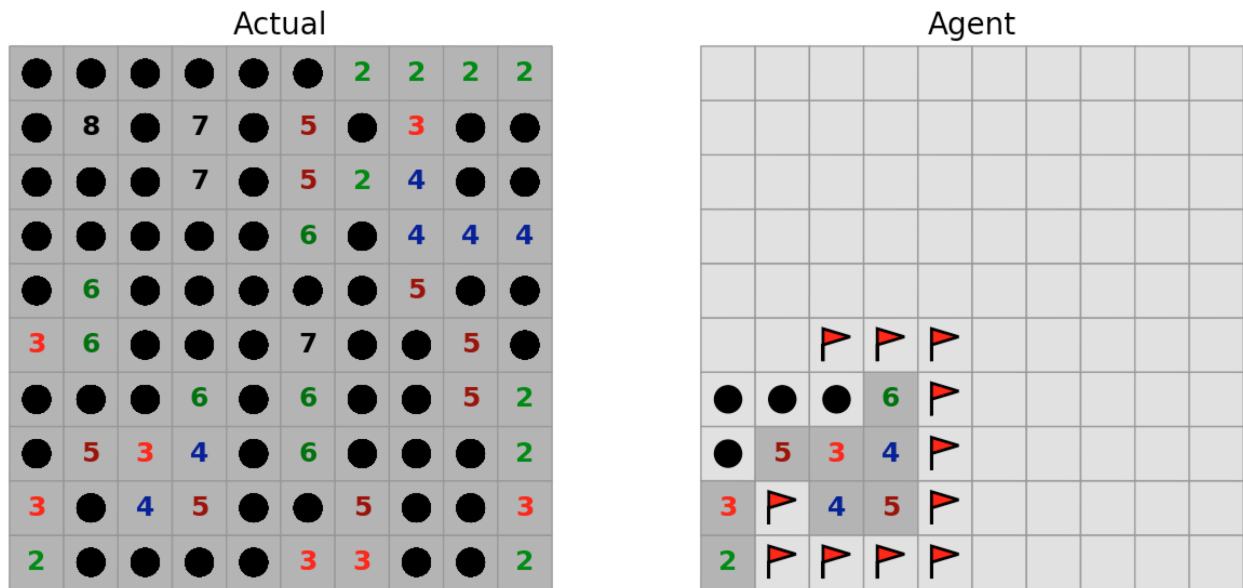


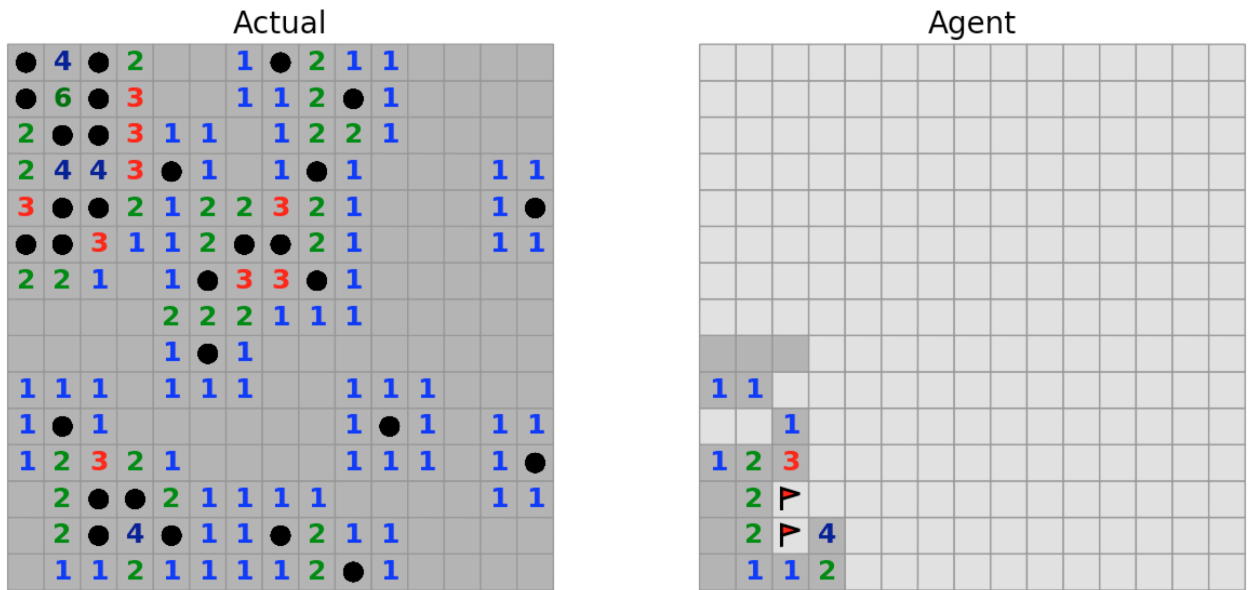
Figure 3: An example where the random opening strategy based on heuristics would get stuck

Our random opening algorithm looks at neighbours of all the open cells and then randomly clicks on the neighbours of an open-cell with the minimum risk value. However, in cases where all the neighbours of open-cells are mines, it will not be able to proceed further. For this we have coded a final phase for our agent - When everything else fails and there is no way of proceeding further just click randomly anywhere - something which we also do when we are stuck.

1.4 Performance

1.4.1 Performance 1

We take a 15x15 environment grid and use 0.13 mine density, which leads us to the following progression.



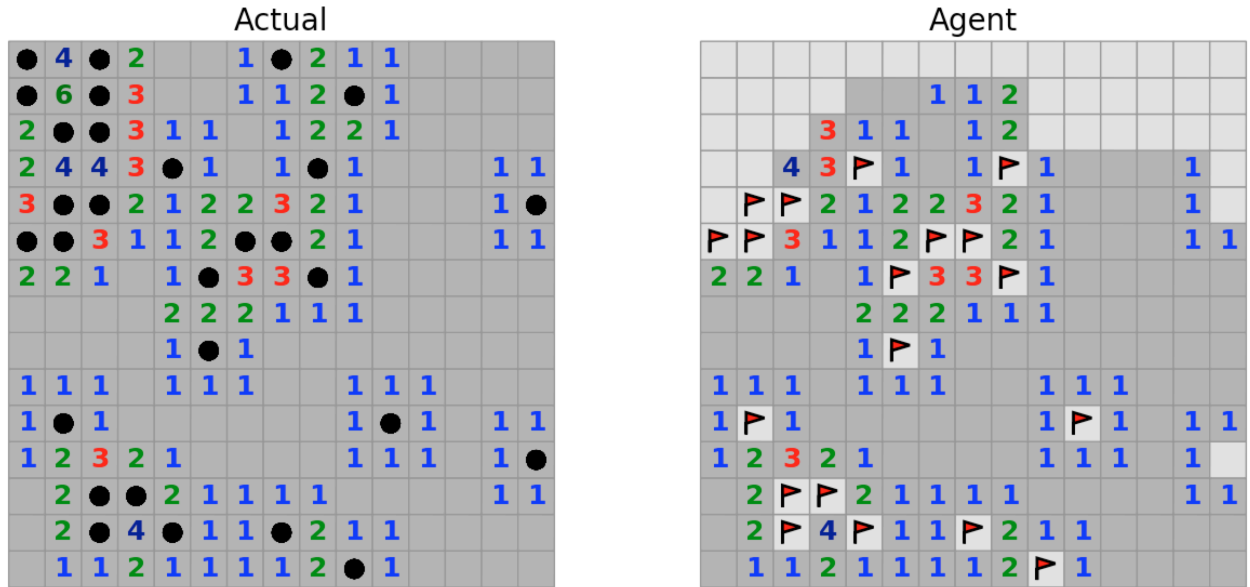


Figure 6: Almost finished playing the game

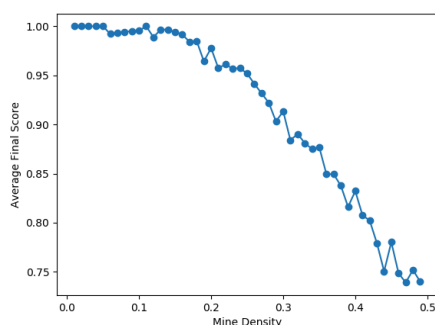


Figure 7: Game over

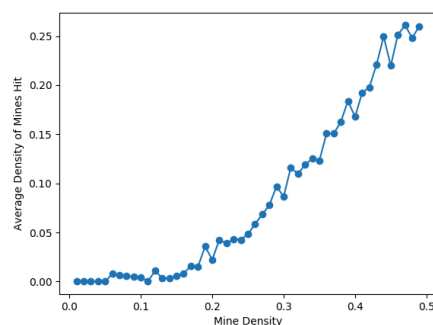
We can observe that in the above game, our agent played very efficiently and was able to flag all the mines properly except the last 2 in the top left corner. It was not a surprise for us because on observing the set of equations that were left at that stage, there were no subsets which could be evaluated and hence no way for the agent to deduce mine and non-mine variables. Technically, we can use backtracking in such cases but we have left it for future work. Hence, based on whatever we coded, the agent just clicked on the cells

randomly.

1.4.2 Performance 2



(a) Average Final Score vs. Mine Density



(b) Average Density of Mines Opened vs. Mine Density

Figure 8: Performance metrics for a 15x15 minesweeper vs. Mine density

The agent plays minesweeper on a board of size 15x15 and for each mine-density, we play 10 games. The mine density ranges from 0.01 to 0.5 with a step-size of 0.01. We observe that the results obtained agree with our intuition.

- As the mine density increases, the average final score decreases. For the first few mine- densities the number of mines flagged is exactly equal to the total number of mines present giving a score of 1.0. However, with increasing mine-density the difficulty also increases and hence the average score goes down.
- Similarly, with increased mine-density the average density of mines hit by the agent also increases. This also agrees with our intuition and we see that although the agent opens more number of mines as the density increases, this increase is gradual and not steep.

Based on the above graphs, we see that the game can be called "hard" when the density of mines hit by the agent is 0.10 - the agent hits 10 percentage of the total mines - which occurs at mine-density 0.3.

1.5 Efficiency

While implementing the minesweeper agent, we did encounter a time constraint

- The knowledge base of the agent is represented by a python list with the individual equations as tuples. When the agent removes a variable from other constraint-equations, it will make a pass through its entire knowledge base. This is a time consuming operation. For very large grid sizes and large number of equations, this leads to increase in time complexity.

The above problem is an implementation-specific constraint which can be removed by changing one aspect of the way in which we store equations. We also store the equations which a

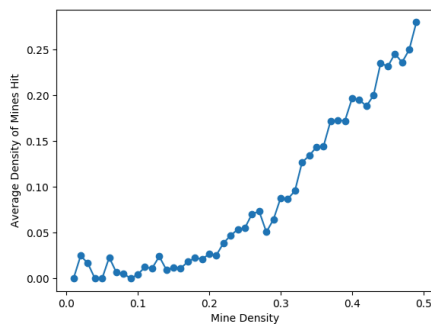
variable is part of as a separate attribute of the variable. Hence, when we need to remove variable, we directly access this attribute containing all the equations which this variable is part of and make a pass through only those specific equations rather than the entire knowledge base.

1.6 Improvements

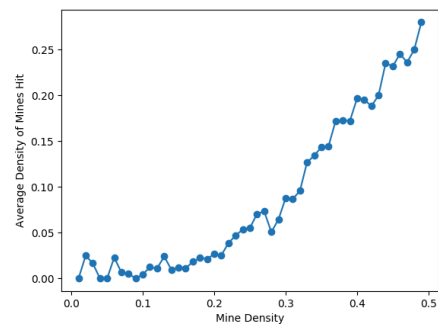
The information about the total number of mines in the environment is modelled in such a way that if the number of cells flagged becomes equal to the total number of mines in the game, then the agent will blindly open all the closed cells without any inferences. This check can be added at various stages in our code to make the solving of the game faster

- In the basic solver phase, the agent makes the check when all the obvious equations have been checked. This means that the agent makes the check at the end of basic solver. If the number of mines present in the game equal the number of mines uncovered, the agent uncovers all other cells and then exits the game.
- In the subset solver phase, the agent makes the check after solving the subsets at every iteration (i.e., before using the created subsets to go for the recursive round). Similarly, if the count found is equal, then the agent uncovers all the cells and exits the game.

This information helps the agent to be computationally more efficient. This is because the agent is able to reach a consensus based on this information, so it saves on checking the remaining cells and just uncovers them and exits. This is efficient compared to the normal CSP agent. Hence, the time to play the game decreases a lot which can be seen by the performance plots below.

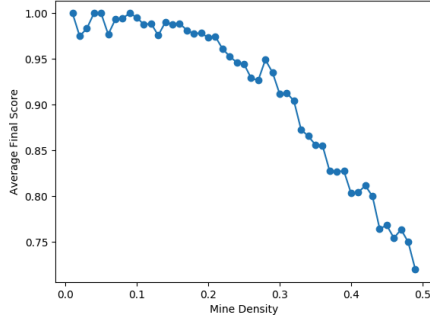


(a) The Improved CSP Agent

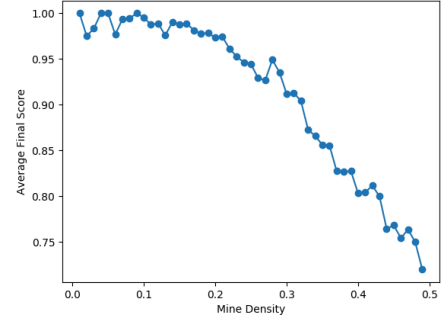


(b) The Normal CSP Agent

Figure 9: Average Density of Mines Hit vs Mine Density

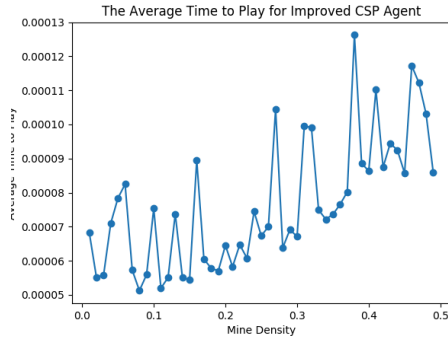


(a) The Improved CSP Agent

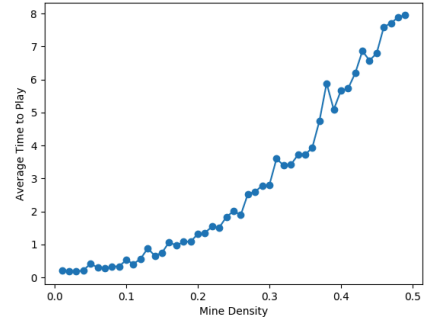


(b) The Normal CSP Agent

Figure 10: Average Final Score vs Mine Density



(a) The Improved CSP Agent



(b) The Normal CSP Agent

Figure 11: Average Game Playing Time vs Mine Density

From the above plots, we observe that the average final score and the average number of mines opened is the same for both the agents. However, there is a very significant difference in the playing times of these agents and the reason is what we mentioned before - the improved agent has an extra knowledge which makes it stop evaluating the equations once the total mines are opened or flagged.

Analysis and Discussion of Results

The agent's performance was evaluated on a 15x15 grid with varying mine densities ranging from 0.01 to 0.5. The results indicate that the agent performs optimally at low to moderate mine densities, achieving near-perfect accuracy in identifying and flagging mines. As the mine density increases, the agent's reliance on random opening strategies grows, resulting in a gradual decline in performance. For densities above 0.3, the game becomes increasingly difficult due to limited actionable information, highlighting the need for advanced techniques like backtracking. Time efficiency analysis shows that the improved agent, with its optimized knowledge base, performs significantly faster than traditional CSP approaches, particularly in scenarios with fewer constraints. These findings underscore the importance of balancing logical inference with probabilistic reasoning to handle uncertainty effectively.

Conclusion

The Smart Minesweeper project demonstrates the successful application of CSP techniques to a challenging decision-making problem. By combining logical inference, subset evaluation, and heuristic strategies, the agent achieves high efficiency and accuracy in solving Minesweeper across a range of conditions. However, its performance is limited in high-density mine scenarios, where backtracking and advanced probabilistic models could provide further improvements. This work lays a foundation for future research in CSPs and decision systems, with potential applications in robotics, automated planning, and other domains requiring intelligent navigation of uncertain environments.

References/Bibliography

1. "Constraint Satisfaction Problems in Artificial Intelligence" - Author(s), Journal, Year.
2. "Heuristic Methods for Game Solving" - Author(s), Conference Proceedings, Year.
3. "Probabilistic Models in Decision Making" - Author(s), Publisher, Year.